

Livrable : API de Prédiction avec FastAPI

Description

L'API FastAPI a été **développée localement sur PyCharm**, où le modèle de segmentation d'images a été intégré et testé. Elle a ensuite été **déployée sur Azure** à l'aide de la technologie Docker. L'API permet de recevoir une image en entrée et de retourner le masque segmenté généré par le modèle. Ce livrable facilite l'utilisation du modèle par Laura et son intégration dans le système global.

Détails des Livrables

- **Lien Azure de l'API déployée :**
<http://fastapi-p8.westeurope.azurecontainer.io:8000/docs>
- **Code source sur GitHub :**
Le code source de l'API, incluant tous les fichiers nécessaires (`main.py`, configuration du modèle, `requirements.txt`, etc.), est disponible dans le dépôt suivant :
https://github.com/preudh/OC_IA_P8_VoitureAutonome

Structure de l'API

L'API est organisée comme suit :

```
api/
├── Dockerfile           # Fichier pour la création de l'image Docker
├── .dockerignore        # Liste des fichiers/dossiers ignorés par Docker
├── main.py              # Script principal de l'API FastAPI
├── models.py            # Gestion et configuration du modèle de segmentation
├── requirements.txt      # Liste des dépendances nécessaires
└── models/              # Répertoire contenant les fichiers du modèle entraîné
```

Technologie de Déploiement

- **Docker :**
L'API a été conteneurisée avec Docker pour garantir une portabilité et une facilité de déploiement. Le fichier `Dockerfile` spécifie les étapes nécessaires à la création de l'image Docker.
- **Azure :**
L'API est déployée sur un conteneur Azure pour fournir un accès public et garantir une haute disponibilité.

Accès et Utilisation

Pour tester l'API, utilisez l'interface interactive générée par FastAPI à l'URL mentionnée ci-dessus. Cette interface permet d'envoyer des requêtes avec une image en entrée et de recevoir la segmentation correspondante.

Axe d'amélioration : CI/CD et MLops

Une **approche CI/CD** pourrait être mise en œuvre pour automatiser et fiabiliser le déploiement :

CI/CD avec GitHub Actions :

1. **Pipeline CI** : Tester et construire automatiquement l'image Docker pour garantir la qualité du code.
2. **Pipeline CD** : Une fois validée, l'image Docker pourrait être poussée vers un registre tel qu'Azure Container Registry (ACR) et déployée automatiquement sur Azure.

Utilisation de plusieurs fichiers Docker Compose :

- `docker-compose.ci.yml` : Pour gérer les tests CI.
- `docker-compose.dev.yml` : Pour le développement local.
- `docker-compose.prod.yml` : Pour le déploiement en production.

Automatisation avec GitHub Actions et Docker Compose :

- Un fichier `.yml` dans GitHub Actions pourrait orchestrer l'ensemble du pipeline CI/CD et utiliser les fichiers Docker Compose pour gérer chaque étape.
- **Avantages** : Meilleure reproductibilité, réduction des efforts manuels, et intégration fluide dans une logique **MLops**. Bien que ce processus n'ait pas été entièrement automatisé ici, l'architecture actuelle reste une base solide pour une future mise en place de cette automatisation.